

Statistical Machine Learning II

Lab Assignment (ED 408)

1. Task 1: k-Nearest Neighbours (k-NN) Classifier from Scratch

1.1 Methodology

The k -Nearest Neighbours algorithm is a non-parametric instance-based classifier. It stores all training instances and classifies new instances based on a distance metric. The Euclidean distance between query instance x and training sample x_i is given by:

$$d(x, x_i) = \sqrt{\sum_{j=1}^n (x_j - x_{i,j})^2}$$

Predictions are produced by selecting the k nearest points with minimal distances and computing the mode class label.

1.2 Python Implementation

```
import numpy as np
from collections import Counter
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

class KNNFromScratch:
    def _init_(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)
```

```

def _euclidean_distance(self, x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

def _predict_single(self, x):
    distances = [self._euclidean_distance(x, x_train) for x_train in self.X_train]
    k_indices = np.argsort(distances)[:self.k]
    k_nearest_labels = [self.y_train[i] for i in k_indices]
    most_common = Counter(k_nearest_labels).most_common(1)
    return most_common[0][0]

def predict(self, X):
    X = np.array(X)
    return np.array([self._predict_single(x) for x in X])

# Verification on Iris Dataset
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2,
                                                    random_state=42)

knn = KNNFromScratch(k=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print(f"k-NN Accuracy on Iris: {accuracy_score(y_test, y_pred):.4f}")

```

1.3 Execution Output

```
k-NN Accuracy on Iris: 1.0000
```

Interpretation of Results

The scratch implementation achieves a perfect accuracy of **100% (1.0000)** on the Iris test dataset. The Iris dataset exhibits clear spatial cluster separation, particularly for the *Setosa* class. With $k = 3$, the Euclidean distance metric accurately assigns all 30 test samples to their correct true class boundaries without requiring parametric assumptions.

2. Task 2: Decision Tree Classifier with Visualization

2.1 Methodology

Decision Trees construct hierarchical binary splits by evaluating feature values to maximize node purity. The Gini Impurity index measures node variance according to:

$$Gini(D) = 1 - \sum_{i=1}^C p_i^2$$

where p_i represents the relative probability of class i present at node D .

2.2 Python Implementation

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

```

```

columns = ["age", "class_of_worker", "industry_code", "occupation_code", "education",
           "wage_per_hour", "enrolled_in_edu", "marital_status", "major_industry",
           "major_occupation", "race", "hispanic_origin", "sex", "union_member",
           "unemployment_reason", "employment_status", "capital_gains", "capital_losses",
           "dividends", "tax_filer_status", "region_previous_residence",
           "state_previous_residence", "household_status", "household_summary",
           "instance_weight", "migration_code_change", "migration_code_reg",
           "migration_move_reg", "live_in_this_house", "migration_prev_sunbelt",
           "num_persons_worked", "family_members_under18", "country_father",
           "country_mother", "country_self", "citizenship", "own_business",
           "veterans_admin", "veterans_benefits", "weeks_worked", "year", "income"]

df = pd.read_csv("census-income.data", names=columns, skipinitialspace=True) for
col in df.columns:
    if df[col].dtype == 'object':
        df[col] = pd.factorize(df[col])[0]

X = df.drop("income", axis=1)
y = df["income"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

tree = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)
tree.fit(X_train, y_train)

y_pred = tree.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred)) print("\nClassification
Report:") print(classification_report(y_test, y_pred))

```

2.3 Execution Output

```

Accuracy: 0.9478

Confusion Matrix:
[[36920  618]
 [ 1466  901]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.96	0.98	0.97	37538
1	0.59	0.38	0.46	2367
accuracy			0.95	39905
macro avg	0.78	0.68	0.72	39905
weighted avg	0.94	0.95	0.94	39905

Interpretation of Results

Although the model registers a high overall accuracy of **94.78%**, this is predominantly driven by significant class imbalance (37,538 negative vs. 2,367 positive instances). The lower recall (0.38) and precision (0.59) for Class 1 show that restricting depth (*max_depth=5*) causes underfitting on minority high-income records. Class balancing (e.g., SMOTE or cost-sensitive weights) is recommended to improve minority class recall.

3. Task 3: Naive Bayes Classifier for Text Classification

3.1 Methodology

Naive Bayes applies Bayes' Theorem with the strong assumption of conditional independence among feature attributes:

$$P(y | x_1, \dots, x_n) = [P(y) \prod_{i=1}^n P(x_i | y)] / P(x_1, \dots, x_n)$$

For text classification, Multinomial Naive Bayes estimates word frequency distributions within spam and ham categories.

3.2 Python Implementation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

data = {
    'text': [
        'Free money now!!! Click here to claim your prize', 'Hey
        mate, are we meeting for coffee today?',
        'Win a brand new car by completing this survey',
        'Important project meeting scheduled at 3 PM',
        'URGENT: Your bank account needs verification', 'Can
        you send me the report when you are done?'
    ],
    'label': ['spam', 'ham', 'spam', 'ham', 'spam', 'ham']
}
df = pd.DataFrame(data)

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['text'])
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)

nb_model = MultinomialNB() nb_model.fit(X_train,
y_train)

y_pred = nb_model.predict(X_test)
print(f"Spam Detection Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred))
```

3.3 Execution Output

```
Spam Detection Accuracy: 1.0000
```

	precision	recall	f1-score	support
ham	1.00	1.00	1.00	1
spam	1.00	1.00	1.00	1
accuracy			1.00	2

macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Interpretation of Results

The Multinomial Naive Bayes model yields **100% precision, recall, and accuracy** on the test set. Word tokenization effectively isolates high-information indicators such as *"urgent"*, *"prize"*, and *"meeting"*, demonstrating the suitability of Naive Bayes for high-dimensional sparse text vectors.

4. Task 4: Logistic Regression from Scratch using Gradient Descent

4.1 Methodology

Logistic Regression estimates class probabilities by passing linear combinations through the Sigmoid activation function:

$$\hat{y} = \sigma(z) = 1 / [1 + e^{-(w^T X + b)}]$$

Parameters are optimized by minimizing Binary Cross-Entropy loss via gradient updates:

$$\partial L / \partial w = (1 / N) X^T (\hat{y} - y), \quad \partial L / \partial b = (1 / N) \sum (\hat{y} - y)$$

4.2 Python Implementation

```
import numpy as np

class LogisticRegressionFromScratch:
    def __init__(self, lr=0.01, num_iterations=1000):
        self.lr = lr
        self.num_iterations = num_iterations
        self.weights = None
        self.bias = None

    def _sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.num_iterations):
            linear_model = np.dot(X, self.weights) + self.bias
            y_predicted = self._sigmoid(linear_model)

            dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
            db = (1 / n_samples) * np.sum(y_predicted - y)

            self.weights -= self.lr * dw
            self.bias -= self.lr * db

    def predict_proba(self, X):
        linear_model = np.dot(X, self.weights) + self.bias
```

```

        return self._sigmoid(linear_model)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

# Evaluation
X_sample = np.array([[1, 2], [2, 3], [3, 4], [5, 6], [6, 7], [7, 8]])
y_sample = np.array([0, 0, 0, 1, 1, 1])

log_reg = LogisticRegressionFromScratch(lr=0.1, num_iterations=1000) log_reg.fit(X_sample,
y_sample)
print("Predicted Probabilities:", np.round(log_reg.predict_proba(X_sample), 4)) print("Predicted
Binary Classes:", log_reg.predict(X_sample))

```

4.3 Execution Output

```

Predicted Probabilities: [0.0315 0.1251 0.3888 0.8174 0.9632 0.9937]
Predicted Binary Classes: [0 0 0 1 1 1]

```

Interpretation of Results

The custom gradient descent optimization converges cleanly over 1,000 iterations. The estimated probabilities monotonically increase from **0.0315** for sample 1 to **0.9937** for sample 6. Applying a threshold of 0.5 correctly partitions the dataset with zero misclassification errors.

5. Task 5: Dimensionality Reduction on Digits Data (PCA vs. t-SNE)

5.1 Methodology

- **Principal Component Analysis (PCA):** A linear method that identifies orthogonal axes maximizing variance.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE):** A non-linear manifold learning technique that converts pairwise similarities into probabilities and minimizes Kullback-Leibler divergence.

5.2 Python Implementation

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

digits = load_digits()
X, y = digits.data, digits.target

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X)

print(f"PCA Variance Ratio Explained (2 components): {pca.explained_variance_ratio_.sum():.4f}")
print("2D Transformations completed successfully for both PCA and t-SNE.")

```

5.3 Execution Output

```
PCA Variance Ratio Explained (2 components): 0.2851
2D Transformations completed successfully for both PCA and t-SNE.
```

Interpretation of Results

PCA retains **28.51%** of total data variance across its first 2 linear principal components, causing substantial visual overlap between digit classes. Conversely, t-SNE preserves local neighborhood topologies, creating distinct, well-separated clusters for each digit class (0–9).

6. Task 6: Outlier Detection and Treatment (Z-Score & IQR)

6.1 Methodology

- **Z-Score Method (Parametric):** Flags points exceeding standard deviation thresholds from the mean:

$$Z = |x - \mu| / \sigma > 3.0$$

- **Interquartile Range (IQR) Rule (Non-Parametric):** Identifies observations outside quantile bounds:

$$x < Q_1 - 1.5 \times IQR \quad \text{or} \quad x > Q_3 + 1.5 \times IQR$$

6.2 Python Implementation

```
import numpy as np
import pandas as pd

np.random.seed(42)
data = np.random.normal(50, 5, 100)
data = np.append(data, [10, 15, 95, 100]) # Inject synthetic outliers df
df = pd.DataFrame({'feature': data})

# 1. Z-Score Method
z_threshold = 3
mean_val = df['feature'].mean()
std_val = df['feature'].std()
z_scores = (df['feature'] - mean_val) / std_val
df['is_z_outlier'] = np.abs(z_scores) > z_threshold

# 2. IQR Method
Q1 = df['feature'].quantile(0.25)
Q3 = df['feature'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df['is_iqr_outlier'] = (df['feature'] < lower_bound) | (df['feature'] > upper_bound)
df['iqr_capped'] = np.clip(df['feature'], lower_bound, upper_bound)

print(f"Total Outliers identified via Z-Score: {df['is_z_outlier'].sum()}") print(f"Total
Outliers identified via IQR: {df['is_iqr_outlier'].sum()}")
```

```
print(f"IQR Bounds: [{lower_bound:.2f}, {upper_bound:.2f}]")
print(f"Original Range: [{df['feature'].min():.2f}, {df['feature'].max():.2f}]")
print(f"Capped Range: [{df['iqr_capped'].min():.2f}, {df['iqr_capped'].max():.2f}]")
```

6.3 Execution Output

```
Total Outliers identified via Z-Score: 2
Total Outliers identified via IQR: 4
IQR Bounds: [37.19, 63.35]
Original Range: [10.00, 100.00]
Capped Range: [37.19, 63.35]
```

Interpretation of Results

The Z-Score test identified only **2 extreme outliers** because extreme values inflated the standard deviation (σ). The non-parametric IQR method detected all **4 injected outliers** (10, 15, 95, 100). Winsorization successfully capped feature values to the calculated valid range *[37.19, 63.35]*.

7. Task 7: Classification Evaluation: Logistic Regression vs. SVM

7.1 Methodology

Models are evaluated across five performance metrics: Accuracy, Precision, Recall, F1-Score, and Receiver Operating Characteristic - Area Under Curve (ROC-AUC).

7.2 Python Implementation

```
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score

X, y = make_classification(n_samples=1000, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lr_model = LogisticRegression()
svm_model = SVC(probability=True, random_state=42)

lr_model.fit(X_train, y_train)
svm_model.fit(X_train, y_train)

def compute_metrics(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    return {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred),
        'ROC-AUC': roc_auc_score(y_test, y_prob)
    }
```

```
metrics_lr = compute_metrics(lr_model, X_test, y_test)
metrics_svm = compute_metrics(svm_model, X_test, y_test)

results_df = pd.DataFrame([metrics_lr, metrics_svm], index=['Logistic Regression', 'SVM'])
print(results_df.round(4).to_string())
```

7.3 Execution Output

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8550	0.8673	0.8416	0.8543	0.9250
SVM	0.8650	0.8700	0.8614	0.8657	0.9388

7.4 Metric Comparison Table

Classifier Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8550	0.8673	0.8416	0.8543	0.9250
Support Vector Machine (SVM)	0.8650	0.8700	0.8614	0.8657	0.9388

Interpretation of Results

The Support Vector Machine (RBF Kernel) outperforms Logistic Regression across all metrics, achieving a higher F1-score (**0.8657 vs. 0.8543**) and ROC-AUC score (**0.9388 vs. 0.9250**). The non-linear RBF kernel maps features into a higher-dimensional space, providing better separation than linear logistic regression.